

Static Binary Rewriting for AFL-style Edge Coverage Instrumentation

Scandroglio Edoardo

September 13, 2026

Contents

1	Introduction	2
2	Motivation	4
2.1	Problem Statement	4
2.2	State of the Art	4
2.3	Goals and Challenges	5
3	Approach	6
3.1	Approach Overview	6
3.2	Approach Details	6
4	Implementation Details	9
4.1	System Architecture	9
4.2	System Details	9
5	Experimental Validation	13
5.1	Goals	13
5.2	Dataset	13
5.3	Experimental Setup	13
5.4	Experiment I - Correctness	13
5.5	Experiment II - Instrumentation Overhead	14
6	Limitations	15
7	Future Works	16
8	Conclusion	17

Chapter 1

Introduction

Fuzzing is nowadays one of the most popular software testing and automated vulnerability discovery techniques. Among all fuzzing techniques, one of the most widely used is grey-box fuzzing, which relies on coverage-guided input mutation. The idea is to discover which portions of the target application are visited during execution, in order to generate inputs that maximize the number of instructions explored and, consequently, the number of vulnerabilities discovered during fuzzing. In particular, the most commonly used approach is to track Edge Coverage, i.e., to keep track of which branches are taken (or not taken). If the source code is not available, adding Edge Coverage instrumentation relies on two main techniques:

- *Static Binary Instrumentation*: rewrites the target application before execution.
- *Dynamic Binary Instrumentation*: rewrites instructions during execution.

Dynamic Binary Instrumentation introduces significant overhead (roughly 100–400%) and, due to this, wastes a considerable amount of resources and time. On the other hand, Static Binary Instrumentation in the absence of source code is difficult, because it is based on modifying a binary without recompiling it. A static binary rewriter must preserve the correctness of the original binary. In this thesis I will focus on Static Binary Instrumentation. The current state-of-the-art could be represented by:

- **Z AFL** is a Static Binary Instrumentation tool that inserts AFL-style edge coverage instrumentation inside a binary while optimizing the code.
- **Retrowrite** : create a "reassembleable assembly" adding labels instead of references and make instrumentation easier, because there is not the risk of breaking these references.

Retrowrite main drawback is that it supports only PIE non-stripped binaries and does not provide an edge coverage instrumentation mechanism, relying in-

stead on afl-gcc. ZAFL, on the other hand, achieves overhead close to compile-time instrumentation, but remains fragile against obfuscated or anti-analysis binaries. This is reported in the paper *Robust, Efficient, and Widely Available Greybox Fuzzing for COTS Binaries with System Call Pattern Feedback*. Moreover, ZAFL instruments the shared-map update as a call to a function, rather than placing the update instructions inline within each basic block.

My approach is to implement an edge coverage instrumentation tool that is compatible with AFL++ and relies on the GRAPE static binary rewriter. The instructions must be placed inline within each basic block and must work on both stripped and non-stripped binaries.

This approach is implemented through two main stages:

- First, I extend the binary by inserting a dynamic dependency to a previously compiled shared object, and by inserting new sections into the binary along with new relocation entries.
- Second, I parse the newly obtained binary with GRAPE and then insert the shared-memory update instructions inline at the beginning of each basic block.

The shared object contains a forkserver that implements the AFL++ communication protocol.

In only one of the conducted tests did the software fail to instrument the target binary. However, on some targets the instrumentation causes a segmentation fault when accessing the .bss section, due to the reorganization of binary segments performed by the lief library used to insert the dynamic dependency. The performance obtained on the tested targets through AFL++ is comparable to that of source-code instrumentation performed by AFL++, and slightly better than ZAFL.

- A static rewriting pipeline that instruments stripped and non-stripped binaries with AFL-style edge coverage without requiring source or symbol information.
- A technique for handling indirect control flow that preserves the correctness of binaries.

Chapter 2

Motivation

2.1 Problem Statement

The problem I aim to solve is to statically instrument a binary with edge coverage instructions in a way that is compatible with the AFL++ fuzzer, that adds as little overhead as possible with respect to the overhead added by compile-time instrumentation, and that preserves the correctness of the binary.

This is important because lower instrumentation overhead means more executions per second during fuzzing, which allows a larger portion of the input space to be explored in the same amount of time, increasing the probability of discovering a given vulnerability.

2.2 State of the Art

The state of the art in terms of fuzzers is represented by the AFL++ fuzzer, which includes state-of-the-art techniques for seed selection, queue culling, and performance. However, AFL++ relies on instrumentation added at compile time. In the absence of source code, the state of the art is represented by several alternatives:

- **AFL in QEMU mode:** A Dynamic Binary Instrumentation approach based on the QEMU virtualized environment. AFL uses QEMU in user-mode emulation. In this mode, QEMU translates guest machine code into intermediate representation blocks called Translation Blocks (TBs). AFL patches this translation mechanism. Every time QEMU translates a new TB, the patching mechanism appends the shared-map update instructions at the end of it, before this code is executed. This technique introduces prohibitive overhead, significantly reducing the number of inputs that can be tested.
- **AFL-Dyninst:** A Static Binary Instrumentation approach based on the Dyninst binary rewriter. The Dyninst instrumentation engine works through

trampolines, which degrade execution performance and introduce overhead due to the required context switches. Moreover, every trampoline call must preserve the context: registers and flags must be saved before the trampoline is taken and restored after it returns.

- **RetroWrite:** This tool performs symbolic lifting on the code, replacing every reference discovered during disassembly with a symbolic label. Doing this allows instructions to be added without modifying the references, achieving reassembleable assembly. While RetroWrite can add ASan instrumentation, it cannot natively add AFL-style edge coverage instructions. To do so, the lifted assembly must be instrumented with the afl-gcc tool in order to produce the patched binary. Moreover, community-reported issues document missing instrumentation points when RetroWrite’s symbolized output is compiled through afl-gcc, caused by a mismatch between RetroWrite’s label-naming convention and afl-as’s basic-block detection heuristics. Finally, RetroWrite only works with PIE, non-stripped binaries.
- **ZAFL:** ZAFL performs a first step of binary analysis and Intermediate Representation (IR) reconstruction. From the IR, it performs live-variable and CFG analyses, based on which it carries out optimization steps capable of optimizing the CFG before actually adding the edge coverage instrumentation. The shared-map update instructions are added as function calls inside the basic blocks, causing performance degradation due to the associated context switches.

2.3 Goals and Challenges

The main goal of this thesis is to develop a tool capable of inline, static edge-coverage instrumentation that preserves binary correctness while achieving performance comparable to compile-time instrumentation.

This approach involves several challenges, mainly related to preserving the binary’s correctness and behavior. Preserving all data/code references is considered a particularly hard problem, especially in obfuscated binaries. In fact, although GRAPE correctly reconstructs all the references, binary layout preservation has been the main challenge, mainly because the binary analysis library tends to modify the binary’s segment layout.

Chapter 3

Approach

3.1 Approach Overview

The overall approach is based on the AFL shared-map update, which can be summarized with the following two instructions:

```
shared_map[current^previous]++;  
current >> 1;
```

where **current** and **previous** are the IDs of the basic blocks between which the transition occurs. A forklserver is injected into the target binary by adding a dynamic dependency to a shared library containing the forklserver's code. The forklserver implements the communication protocol with the fuzzer.

The overall instrumentation approach can be divided into the following steps:

1. Modify the ELF by adding a dynamic dependency, new sections, and new relocations.
2. Analyze the new ELF, reconstruct the CFG, and discover the main address.
3. Insert the initial instrumentation at **main**, needed to initialize previous basic block ID.
4. Search for instrumentation points.
5. Instrument the previously found instrumentation points.
6. Adjust the internal references of the ELF.
7. Write the patched file.

3.2 Approach Details

- **Step 1 – Dynamic dependency, sections, and relocations insertion:** Add a dynamic dependency to the shared object *libforkserver.so*.

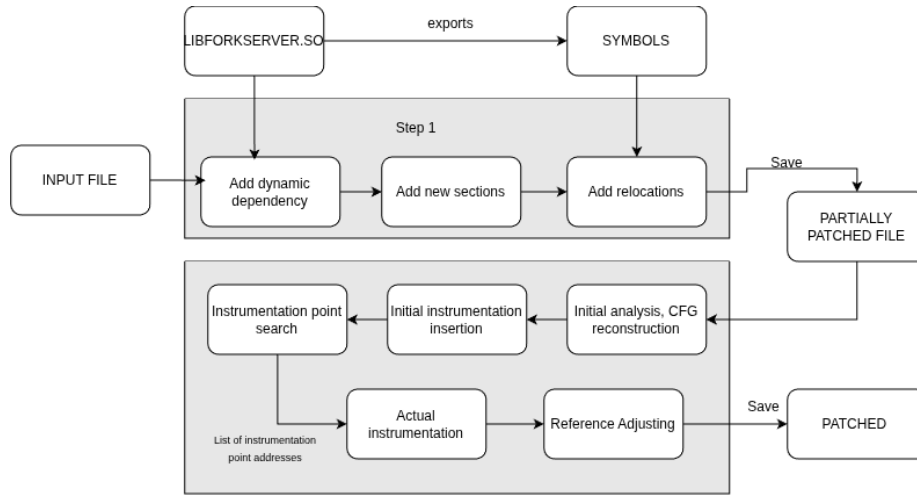


Figure 3.1: Overall Functioning

This shared object contains the forkingserver and implements the communication protocol with AFL/AFL++. Moreover, during this step, two new sections are added to the binary:

- *.mygot*: contains the relocated symbols from the shared object. This is a pointer to the shared map initialized in the libforkserver.so object and a pointer to forkingserver function;
- *.mydata*: a section used as temporary variable storage. It contains space to save the `rax` register, the value of the previous basic block ID, and the dereferenced address of the shared map.

- **Step 2 – Initial analysis:** The CFG is reconstructed, and by analyzing it the main address can be found in both the stripped and non-stripped cases. The addresses of the functions that should not be instrumented (such as `frame_dummy` and other setup functions) are found and excluded from later instrumentation.
- **Step 3 – Initial instrumentation:** Through the relocated symbols, the address of the forkingserver function is found and the forkingserver is invoked. The previous basic block ID is initialized to a random value and stored into the *.mydata* section and then the address of the shared map is obtained from the relocated symbol and stored, for efficiency reasons, into *.mydata*. The assembly instrumented in this initial part can be summarized as follows:

Listing 3.1: Initial Instrumentation Code

```

1 | push rsi

```

```

2 push rdi
3 mov rax, QWORD PTR[rip + offset_to_relocated_forkserver_function]
4 mov rax, [rax]
5 call rax
6 mov WORD PTR[rip + offset_to_previous_saved_ID], random_int
7 lea rax, [rip + offset_to_relocated__afl_area_ptr]
8 mov rax, QWORD PTR [rax]
9 mov rax, QWORD PTR [rax]
10 mov QWORD PTR[rip + offset_to_saved_shared_memory_address], rax

```

- **Step 4 – Instrumentation point search:** Starting from the main address, a Depth-First Search is executed to obtain all the basic blocks reachable from it. Every time a new basic block is found, before it is inserted into the list of basic blocks that need to be instrumented, it is checked against the previously determined list of functions to be excluded from instrumentation. It is also checked whether the last instruction of the previous basic block is a call. Since a call is always taken, the following basic blocks are not instrumented.
- **Step 5 – Actual instrumentation:** For each basic block previously found, its first address is instrumented. Before actually inserting the instructions, the offset to *.mydata* has to be computed relative to the instrumentation address. The inserted instructions can be summarized as follows:

Listing 3.2: Shared Map Update Code

```

1 mov QWORD PTR[rip + offset_to_saved_rax], rax
2 movzx rax, WORD PTR[rip + offset_to_previous_saved_ID]
3 xor ax, random_current_basic_block_ID
4 add rax, QWORD PTR[rip + offset_to_saved_shared_memory_address]
5 inc BYTE PTR[rax]
6 mov WORD PTR[rip + offset_to_precedent_bb_saved], right_shifted_current_ID
7 mov rax, QWORD PTR[rip + offset_to_saved_rax]

```

This snippet of assembly performs the aforementioned shared-map update:
shared_map[current^previous]++;
current>>1;

- **Step 6 – Reference adjustment:** All the instructions of each basic block are scanned in search of displaced references, which, when found, are adjusted to their new, correct value. Then is checked (and eventually corrected) any mismatch between the ELF layout and the ELF header
- **Step 7 – Write the new patched ELF**

Chapter 4

Implementation Details

4.1 System Architecture

There are two main files:

- **instrumentator.py**: contains all the logic used to instrument the target application.
- **libforkserver.so**: This library contains the communication protocol used with the fuzzer, along with the forkserver. It can work both in standalone mode (without an actual fuzzer) and in normal mode. In standalone mode, the map is initialized as a local memory region instead of a shared memory region. The library also exports the symbols to be relocated inside the target application.

4.2 System Details

- **instrumentator.py**: For *Step 1*, the lief Python library is used to add the dynamic dependency, the new sections, and the relocations. For all the other steps, the GRAPE framework is used, which provides API calls to add, remove, and replace an instruction at a specific address in the .text section of the ELF.
- **libforkserver.so**: The library contains a constructor executed before the target's `main`. During the execution of the constructor, it accesses two environment variables to configure the mode in which the forkserver must run. Specifically:
 - `__AFL_SHM_ID`: set by the fuzzer, contains the ID of the shared memory.

- `__AFL_PLUSPLUS`: equal to one if the AFL++ communication protocol is to be used; otherwise, the AFL communication protocol is used.

The forkingserver is executed by the main of the target. Three different function pointers could be used to run the forkingserver in different mode:

- *standalone mode*: The shared map is allocated locally and there is no communication with the fuzzer.
- *AFL mode*: The shared map is obtained through the shared memory ID from the environment variable.
- *AFL++ mode*: Same as the previous, but the communication protocol is different.

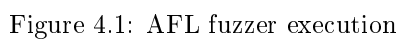
Communication with the fuzzer happens unidirectionally through two different file descriptors created by the fuzzer:

- *forkserver_read (fd 198)*: used by the forkingserver to read a message from the fuzzer.
- *forkserver_write (fd 199)*: used by the forkingserver to write a message to the fuzzer.

Inside libforkserver.so the `__afl_area_ptr` and `__forkserver_start_ptr` symbols are declared with `__attribute__((visibility("default")))`, and therefore visible outside the library. This symbols are then relocated inside the target, where they are read/updated by it.

- `__afl_area_ptr`: contains the address of the shared map that needs to be updated by the target.
- `__forkserver_start_ptr`: contains the pointer to the forkingserver function selected during constructor execution of the shared object.

The communication protocol differs slightly between the AFL and AFL++ modes. During fuzzer execution, the interaction between fuzzer, target, and forkingserver can be represented as follows.



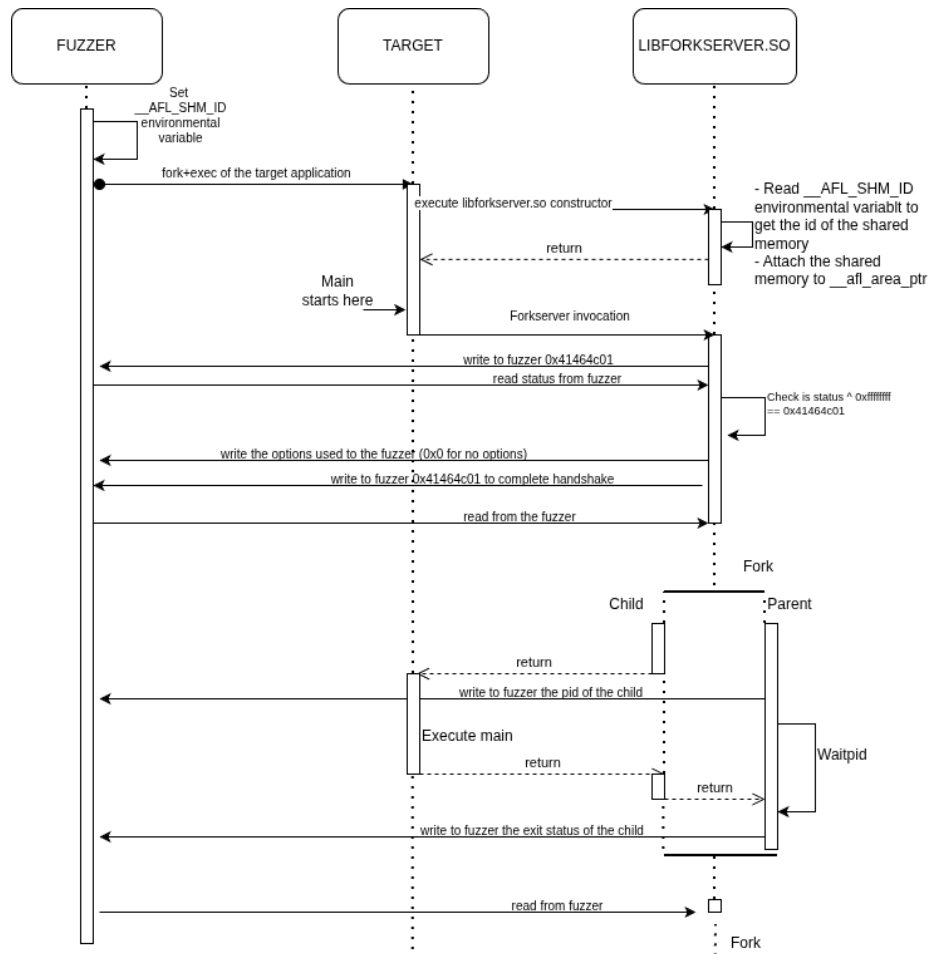


Figure 4.2: AFL++ fuzzer execution

Chapter 5

Experimental Validation

5.1 Goals

- **Goal I** : Show that the rewriter produce correct binaries
- **Goal II** : Show taht instrumentation overhead is competitive with other binary only rewriters and lower than dynamic binary translator.

5.2 Dataset

The dataset used for testing is composed of the original LAVA-M testbench binaries (*base64*, *md5sum*, *uniq*, *who*) already injected with synthetic vulnerabilities. All the binaries are compiled with GCC 5.4.0, all 4 unstripped. The *md5sum* has been downloaded already corrupted. Both afl in qemu mode and the tool developed fails to instrument it. Onlu ZAFL success in instrument it mainly because ZAFL perform an optimization step that probably fix the error.

5.3 Experimental Setup

All the experiments are done with AFL++ version 5.03c and made across 24 hours fuzzing campaign with 3 different repetitions per binary tested.

5.4 Experiment I - Correctness

For *base64* and *uniq* binaries the behavior is preserver after the instrumentation. Running these 2 binaries in standalone mode produce the same output as using the non instrumented version.

who binary instead fail to preserve behavior. Infact the instrumentation cause a Segmentation Fault at each run.

5.5 Experiment II - Instrumentation Overhead

		afl			mine			zafl		
		run1	run2	run3	run1	run2	run3	run1	run2	run3
base64	execs_per_sec	398.49	444.98	351.77	1703.45	1738.99	1842.27	1577.45	1792.35	1749.11
	execs_done	34429329	38446221	30392855	147177902	150249117	159172058	136291378	154859292	151123034
	edges_found	690	690	690	358	370	358	640	672	672
	saved_crashes	7	3	2	12	41	11	10	115	118
md5sum	execs_per_sec	x	x	x	x	x	x	515.09		
	execs_done	x	x	x	x	x	x	44503697		
	edges_found	x	x	x	x	x	x	9		
	saved_crashes	x	x	x	x	x	x	846		
uniq	execs_per_sec	567.81	576.57	577.17	1758.61	1399.39	1864.43	1773.92	1559.98	1605.66
	execs_done	49058377	49815928	49867759	151943556	120907581	161086939	153267103	134782333	138729055
	edges_found	473	473	473	242	242	242	443	443	443
	saved_crashes	2	0	0	0	0	0	0	1	0
who	execs_per_sec	225.11			x	x	x	456.42		
	execs_done	19449595			x	x	x	39434809		
	edges_found	13770			x	x	x	14044		
	saved_crashes	0			x	x	x	2		

Table 5.1: Fuzzing statistics over 24 hours fuzzing campaign on LAVA-M dataset

The empty cells are there because, since there is not a comparison with the tool discussed in this thesis, the experiment had not been done

	base64		uniq		who	
	mine	zafl	mine	zafl	mine	zafl
file_size[MByte]	8,25228	0,137459	8,256376	0,149747	8,62092	1,718515
avg_execs_per_sec	1761.57	1706.3	1674.14	1646.52	x	456.42

Table 5.2: Instrumented binaries statistics

Chapter 6

Limitations

The main limitation is that the developed software fails to preserve the correctness of the target binary when the lief library changes the ELF layout, for example by altering the segment layout. For instance, when the dynamic dependency is added to the binary, some of the sections are modified. Specifically:

- *.dynstr*: the string 'libforkserver.so' has to be placed inside the dynamic string section.
- *.dynamic*: lief adds a DT_NEEDED entry that points to the newly added string in *.dynstr*.

If, during either of these two additions, lief exceeds the available space of the segment containing the grown section, it reorganizes the segments. This causes a complete displacement of the data sections (such as *.bss*, *.data*, *.rodata*, etc.) without a corresponding fix to the instructions that reference these data sections. Another limitation is that this pipeline relies heavily on the Linux ELF binary format and fails to instrument Windows PE binaries.

Chapter 7

Future Works

Improve binary correctness after instrumentation in such a way that the original behavior is preserved. Moreover some of the state-of-the-art techniques that mitigate hash collision could be implemented into the framework in order to improve fuzzing effectiveness.

Chapter 8

Conclusion

The goal of this thesis was to develop a static binary instrumentator that uses GRAPE as binary rewrite framework capable of inserting AFL-style edge coverage instructions inline within each basic block without relying of source of or symbol table, while remains compatible with AFL++ fuzzer. This instrumentation has to add as less overheads as possible with respect to compile time instrumentation.

The developed approach rely on *lief* python library to add needed sections, libraries and relocation and GRAPE to actually insert edge-coverage instrumentation while reconstruct references, both in PIE and nonPIE bineries.

The experiments carried out on LAVA-M testbench shows that the approach is quite capable of correctly instrument the binaries, with 1 on 4 test failing to be instrumented, mainly due to a binary corruption already present. The overhead introduced by the injected instrumentation, when measured through AFL++ execution throughput, is comparable to that of compile-time instrumentation and slightly lower than the overhead introduced by ZAFB, confirming that placing the shared-map update inline, instead of behind a function call, is an effective way of reducing the performance cost of static rewriting.

The main limitation encountered depends on the fact that *lief* python library does not guarantee the preservation of binary segments layout, causing the loss of references to the data sections before the actual instrumentation.

The results obtained shows that Static Binary Instrumentation could be a game changer in fuzzing binary only applications, due to small overheads, bringing binary-only fuzzing closer to source-code-available fuzzing.